



Universidad
Indoamérica

UNITAT 3 · FONAMENTS DE SISTEMES OPERATIUS

Fitxers i sistemes de fitxers

Universidad Tecnológica Indoamérica

Laboratori TD2

Com s'accedeix als fitxers? Dos nivells

Al disc (persistent)

- Estructura del sistema de fitxers: superbloc, taules d'inodes, blocs
- Inodes: les metadades de cada fitxer
- Directoris: llistes *nom* → *inode*

Al nucli (mentre està obert)

- Taula de descriptors, una per procés
- Taula de fitxers oberts, una per a tot el sistema
- Taula d'inodes a la memòria (i la memòria cau de pàgines)

Estructura del sistema de fitxers



disc NVMe →

ext4: el grup de blocs es repeteix al llarg de la partició (amb còpies del superbloc)

Sortida real

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .  
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on  
/dev/nvme0n1p5 ext4 25600000 2372366 23227634   10% /  
block size: 4096 bytes
```

Un fitxer = contingut + inode

- L'**inode** desa: tipus i permisos, enllaços durs, mida, dispositiu, UID/GID, marques de temps i on són els blocs
- **No** desa el nom: el nom és al directori
- **Correcció a l'original:** `ctime` no és la creació sinó l'últim *canvi de l'inode*. La creació (*Birth*) existeix a ext4 i es llegeix amb `statx()`.

Sortida real

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096
regular file
Device: 259,5   Inode: 12190686   Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: (
1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

De l'inode als blocs: punters i extents

inode → extent tree (depth 1, 340 entries per block)

	lògic	longitud	físic
#0	0	32.768	34.816
#1	32.768	32.768	67.681
#2	65.536	32.768	100.546
#3	98.304	32.768	133.411
#4	131.072	32.768	166.276
#5	163.840	9.111	199.141

bloc físic =
 $199.141 + (172.950 - 163.840)$
= 208.251

blocs d'índexs llegits: 1

el mateix fitxer amb punters (ext3):

172,951 punters en 170 blocs d'índexs · aquesta posició: 2 lectures d'índexs · amb extents: 6 × 12 bytes

676 MiB



el fitxer en blocs lògics (0 ... 172,951); cada franja és un extent

Un nom apunta a un inode

Sortida real

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student  9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

Enllaç dur (`link`)

Una altra entrada al **mateix** inode: mateix número, `nlink = 2`. No a directoris ni entre sistemes de fitxers.

Enllaç simbòlic (`symlink`)

Un inode **nou** el contingut del qual és un camí (9 bytes: `notas.txt`). Pot quedar penjat.

Resoldre camins, enllaçar i esborrar

\$ cat atajo

camí relatiu: comença al directori actual, inode 1402

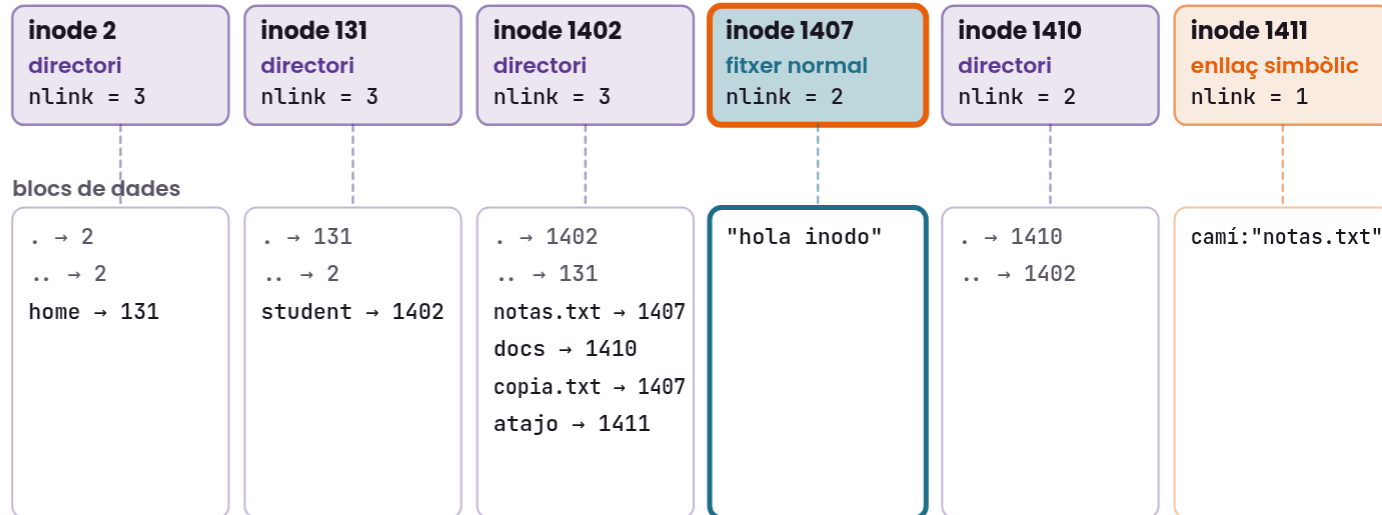
llegir el dir. 1402: "atajo" → inode 1411

enllaç simbòlic núm. 1: continuar amb "notas.txt"

llegir el dir. 1402: "notas.txt" → inode 1407

llegir els blocs de dades de l'inode 1407: "hola inodo"

taula d'inodes



"hola inodo"

Llegir l'inode: stat, fstat, lstat

C

```
int stat(const char *path, struct stat *st);
int fstat(int fd, struct stat *st);
int lstat(const char *path, struct stat *st); /* does not follow links */

struct stat {
    dev_t     st_dev;      ino_t     st_ino;
    mode_t     st_mode;    nlink_t  st_nlink;
    uid_t     st_uid;      gid_t     st_gid;
    dev_t     st_rdev;     /* device, if special file */
    off_t     st_size;     blkcnt_t st_blocks;
    blksize_t st_blksize; /* preferred I/O block size */
    struct timespec st_atim, st_mtim, st_ctim;
};
```

- 0 si va bé; -1 i `errno` si no:
- `EACCES` sense permís de cerca (`x`) en un directori del camí
- `ENOENT` no existeix · `ENOTDIR` un component no és un directori
- `EL00P` més de 40 enllaços simbòlics
- 2026: temps amb nanosegons (`st_mtime = st_mtim.tv_sec`); `statx()` hi afegeix la data de creació.

El tipus és a `st_mode` : macros `S_IS...`

<code>S_ISREG</code>	fitxer normal	-
<code>S_ISDIR</code>	directori	d
<code>S_ISLNK</code>	enllaç simbòlic	l
<code>S_ISCHR</code> ·	dispositiu de caràcters	c ·
<code>S_ISBLK</code>	· de blocs	b
<code>S_ISFIFO</code> ·	canonada · sòcol	p ·
<code>S_ISSOCK</code>		s

Sortida real

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288
bytes, mode 0755)
```

Sortida real

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes,
mode 0600)
```

Sortida real

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584
bytes, mode 0775)
```

Permisos: `access` ... i el seu parany

C

```
int access(const char *path, int mode); /* R_OK W_OK X_OK F_OK */
```

- -1 si no existeix o es denega el permís: `EROFS` (només lectura), `ETXTBSY` (executable en ús)...
- Comprova amb l'UID *real*, no amb l'efectiu

Cursa TOCTOU (actualització 2026)

Entre `access()` i `open()` un altre procés pot canviar el fitxer per un enllaç a `/etc/passwd`. La manera segura és **obrir i gestionar l'error** (`EACCES`), o `openat2()` amb `RESOLVE_NO_SYMLINKS`. El TD8 (`agentd`) fa servir `realpath` i Landlock per aquest motiu.

Crear, enllaçar i esborrar noms

C

```
int mknod(const char *path, mode_t mode, dev_t dev);
int mkfifo(const char *path, mode_t mode); /* S_IFIFO */
int link(const char *old, const char *new);
int symlink(const char *target, const char *path);
int unlink(const char *path); /* EISDIR */
int rmdir(const char *path); /* must be emp
```

- `mknod` crea qualsevol tipus: el tipus es combina amb OR amb els permisos (`S_IFREG|S_IRUSR|S_IXUSR = S_IFREG|0500`); avui es fa servir gairebé només per a dispositius. Per a fitxers: `open(O_CREAT)`; per a canonades: `mkfifo`.
- Els permisos demanats passen per l' `umask` del procés (normalment `022` o `002`).
- `unlink` elimina una *entrada*, no el fitxer.

Obrir o crear: `open`

C

```
int open(const char *path, int flags, ... /* mode_t mode */);  
int openat(int dirfd, const char *path, int flags, ...);
```

- Accés: `O_RDONLY`, `O_WRONLY`, `O_RDWR`
- Creació: `O_CREAT`, `O_EXCL` (falla si existeix), `O_TRUNC`
- Estat: `O_APPEND` (cada `write` al final, de manera atòmica)
- Retorna el **descriptor lliure més baix** (≥ 0) o `-1`

Avui també

- `O_CLOEXEC` : no heretar el descriptor en un `exec` (glibc sempre el fa servir, mira-ho a `strace`)
- `O_NOFOLLOW`, `O_DIRECTORY`, `O_TMPFILE`
- glibc implementa `open` amb `openat(AT_FDCWD, ...)`; `creat()` és `open(..., O_WRONLY|O_CREAT|O_TRUNC)`

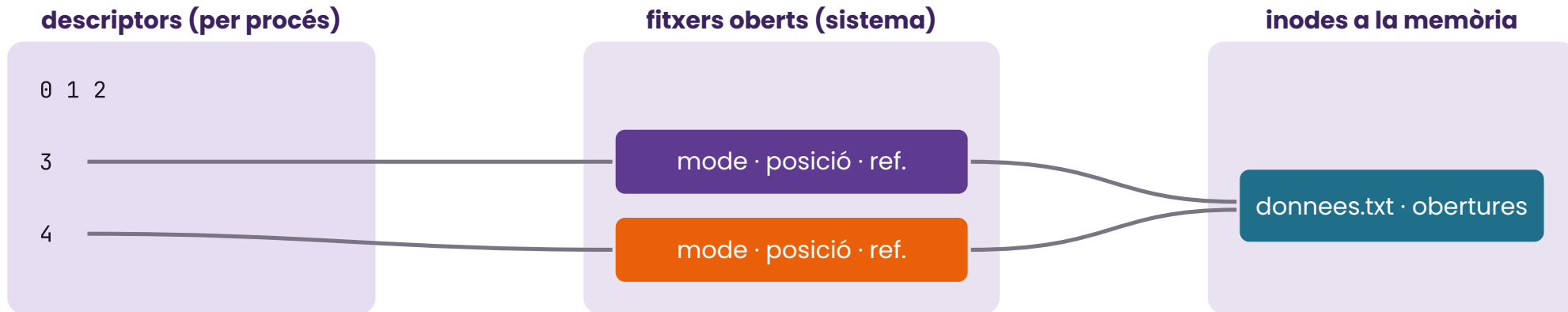
close : quan desapareix un fitxer?

Quan `nlink = 0` i cap descriptor no el té obert. Fins aleshores, esborrat però viu:

Sortida real

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```

Gestió de fitxers al nucli: tres taules



La **posició** viu a la taula del mig: per això importa qui comparteix cada entrada.

Àlies: dues crides `open`, `dup` i `fork`

```
fd = open("donnees.txt", O_RDONLY);
read(fd, buf, 4); write(1, buf, 4);
pid = fork();
/* child */ read(fd, buf, 4); write(1, buf, 4);
/* parent */ wait(NULL); n = read(fd, buf, 4);
```

`read(3)` fa servir l'entrada A: llegeix "efgh" i la seva posició passa a 8

sortida estàndard:
"abcdefgh"

descriptors (per procés)

pare

0	→ terminal
1	→ terminal
2	→ terminal
3	
4	

fill

0	→ terminal
1	→ terminal
2	→ terminal
3	
4	

fitxers oberts (sistema)

entrada A

O_RDONLY
offset = 8
refs = 2

inodes a la memòria (sistema)

donnees.txt

obert per 1 entrada/es

a	b	c	d	e	f	g	h
---	---	---	---	---	---	---	---

Funcions bàsiques, `read` i `write` : repàs

- Inodes: `stat`, `chmod`, `chown`, `link`, `unlink` ...
- Descriptors: `open`, `read`, `write`, `lseek`, `close`, `fcntl` ...
- `fopen` / `fread` : biblioteca, no crides al sistema (`fileno()`)

`read` pot retornar menys bytes, 0 al final, `-1/ EINTR`; `write` pot ser parcial. L'exemple de la diapositiva 51 tenia un error: A2.4.

Recórrer directoris: `opendir`, `readdir`, `closedir`

C

```
DIR *opendir(const char *name);
struct dirent *readdir(DIR *dir); /* NULL at the end */
int closedir(DIR *dir);

struct dirent {
    ino_t      d_ino;      /* inode number */
    unsigned char d_type;  /* DT_REG, DT_DIR, DT_LNK... */
    char       d_name[256];
    /* d_off, d_reclen: internal */
};
```

- `readdir` retorna les entrades en l'ordre intern del sistema de fitxers (a ext4, l'ordre d'un índex de dispersió), no alfabètic
- Inclou `.` i `..`
- **Actualització:** l'original deia «no feu servir els quatre primers camps». Avui `d_ino` i `d_type` són útils (estalvien un `stat`), però `d_type` pot ser `DT_UNKNOWN`: llavors cal `lstat`.

Recórrer un directori: `myls -lR`

DIR *d = opendir(dir); i després:

C tds/td2/myls.c

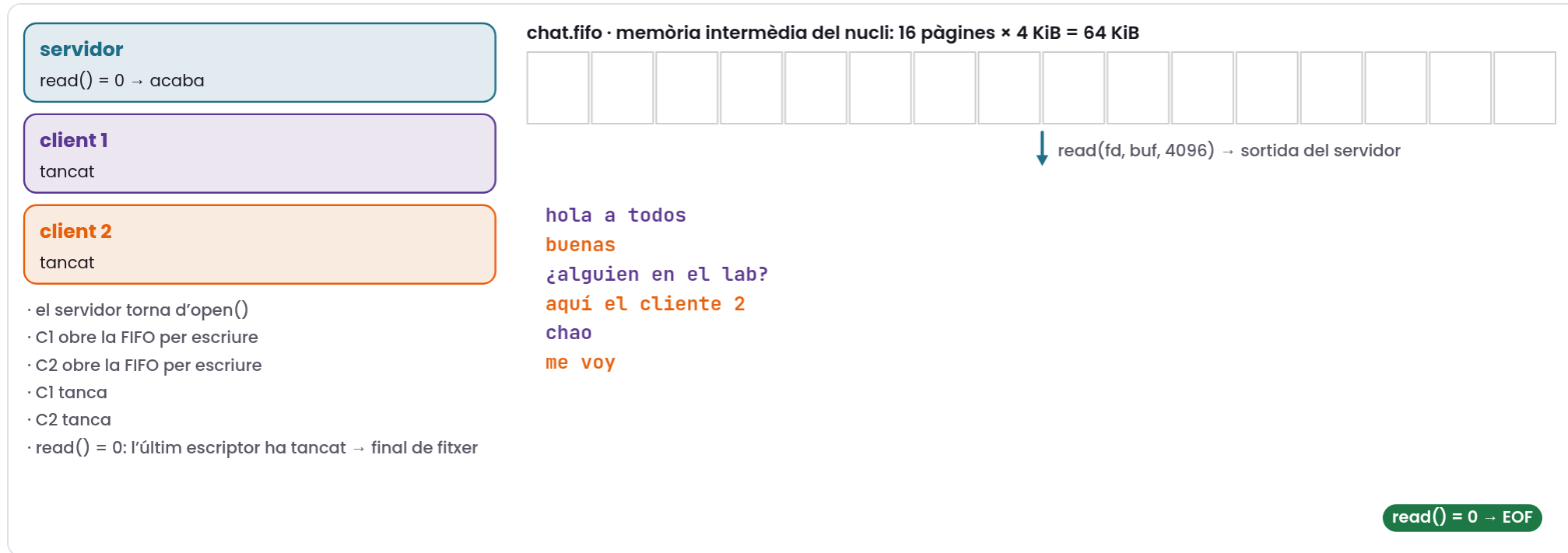
```
struct dirent *e;
while ((e = readdir(d)) != NULL) {
    if (n == cap) {
        cap *= 2;
        names = realloc(names, cap * sizeof *names);
        exit_if(names == NULL, "realloc");
    }
    names[n] = strdup(e->d_name);
    exit_if(names[n] == NULL, "strdup");
    n++;
}
exit_if(closedir(d) == -1, "closedir");
```

Sortida real

```
$ ./myls -lR demo
demo:
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
-rw-rw-r--   1 student student     3 2026-09-30 13:12
a.txt
lrwxrwxrwx   1 student student     5 2026-09-30 13:12
link -> a.txt
prw-rw-r--   1 student student     0 2026-09-30 13:12
pipe
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
sub

demo/sub:
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
```

Un fitxer especial: la FIFO del xat



El que l'original no va arribar a explicar

Registre de diari (journaling)

ext4 i XFS escriuen primer en un diari: després d'un tall de corrent no cal revisar tot el disc.

Còpia en escriptura

Btrfs i ZFS mai no sobreescriuen: instantànies barates i sumes de verificació de cada bloc.

Memòria cau i `fsync`

`write` acaba a la memòria cau de pàgines; només `fsync` garanteix que ha arribat al disc.

«Tot és un fitxer»: `/proc`, `/sys`, `/dev` i FUSE fan servir la mateixa interfície `open` / `read` / `write`.

TD2 resol't + repte IA

- `filetype` amb el terminal, un directori, un fitxer i una canonada
- El xat amb FIFO, amb dos terminals i després tres
- `alias_open` davant d' `alias_dup` ; `myls -lR`

Bash

```
make -C tds && tds/tests/run.sh td2
```



Universidad
Indoamérica

PER EMPORTAR-TE

Tres preguntes

1. On és el nom d'un fitxer, i on és la seva mida?
2. Per què `dup` imprimeix «abcdefgh» i dues crides `open` «abcdabcd»?
3. Quan allibera el nucli els blocs d'un fitxer esborrat?